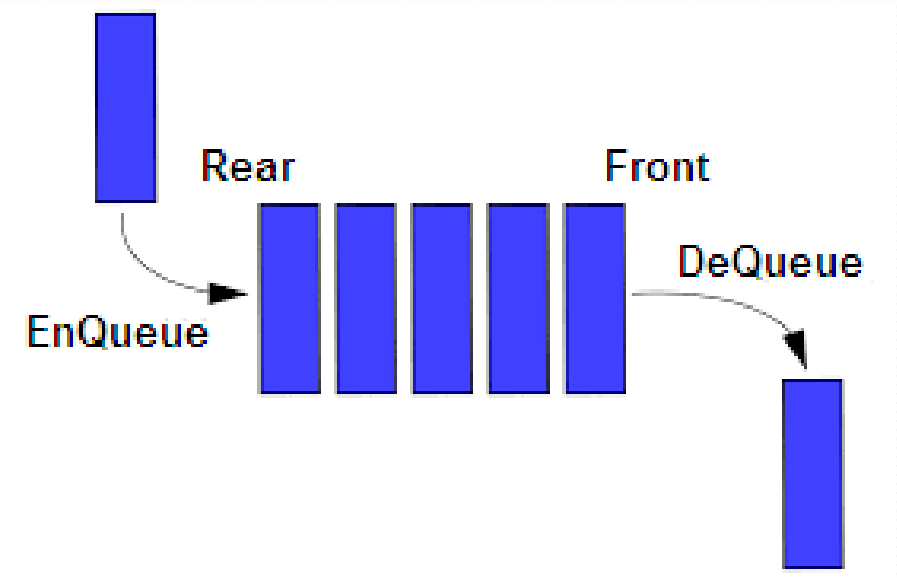
Queue

- Shital Dongre
- Assistant Professor
- VIT, Pune.

What is a Queue?

- Linear data structure
- Queue Principle: FIRST IN FIRST OUT
- Access set of elements in FIFO order
- It means the first element inserted is the first one to be removed
- Ex- waiting queue for bus-first one in line is the first one to get entered in bus

- FIRST IN FIRST OUT



Applications of Queue

- Job scheduling
- Resource scheduling
- Graph traversing algorithm like for Breadth first search

Operations on Queue

- isEmpty
- EnQueue
- DeQueue
- isFull

Queue using array

- #define MAX_QUEUE_SIZE 100
- int front, rear = -1
- int queue[MAX_QUEUE_SIZE]

```
void isEmpty()
{
        if(front==rear)
        {
            printf("\n Queue is empty");
        }
}
```

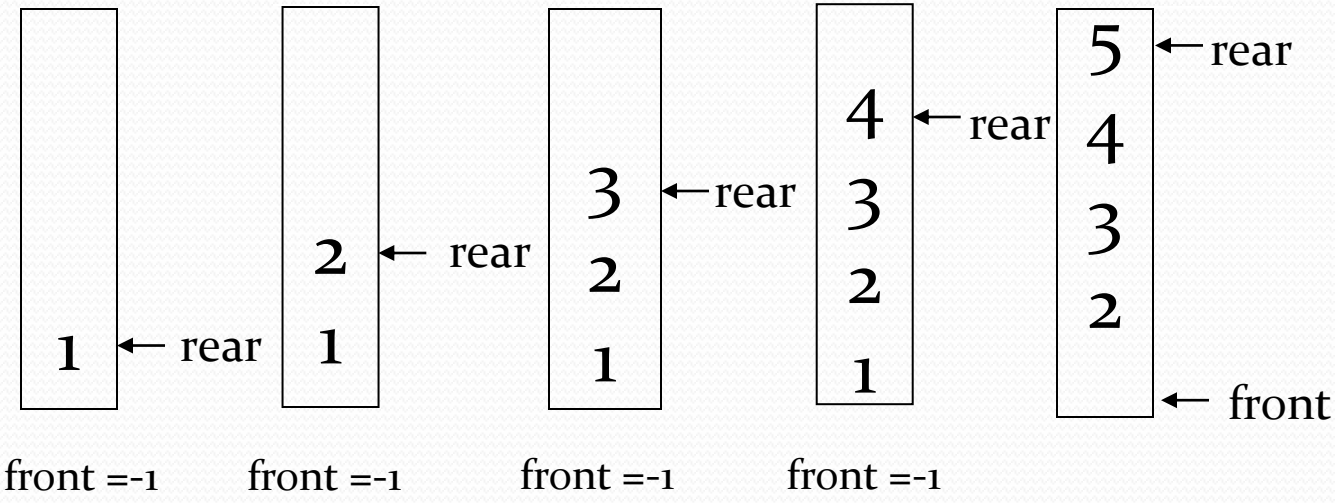
```
int isFull()
{
        if(rear== (MAX_QUEUE_SIZE-1))
         printf("\n Queue is Full");
}
```

```
void EnQueue( int num)
{
        if(rear== (MAX_QUEUE_SIZE-1))
            printf("\n Queue is Full");
         else
         {
           rear = rear + 1;
           Queue[rear] = num;
         }
}
```

```
void DeQueue()
{
        if(front==rear)
         {
            printf("\n Queue is empty");
         }
         else
         {
              front ++;
      printf("\n Deleted Element is %d", queue[front]);
          }
}
```

```
void Q_Display
{
        printf("\n Queue Elements are:\n ");
        if(front==rear)
            printf("\n Queue is Empty");
        else
        {
            for(i=(front+1); i<=rear; i++)
            {
                printf("\n%d",queue[i]);
             }
}
```

First In First Out



Queue using Linked list

```
struct Node
{
    int data;
    struct Node *next;
};

struct Queue
{
    struct Node *front, *rear;
};
```

```
struct Queue *createQ()
{
struct Queue *Q = (struct
Queue*)malloc(sizeof(struct Queue));

    Q->front = Q->rear = NULL;
    return Q;
}
```

```
void enQueue(struct Queue *Q, int data)
{
        struct Node *temp = (struct Node*)
malloc(sizeof(struct Node));

    if (Q->rear == NULL)
    {
        Q->front = Q->rear = temp;
        return;
    }

    Q->rear->next = temp;
    Q->rear = temp;
}
```

```
struct QNode *deQueue(struct Queue *Q)
{

    if (Q ->front == NULL)
       return NULL;


    struct Node *temp = Q ->front;
     Q ->front = Q ->front->next;


    if (Q ->front == NULL)
        Q ->rear = NULL;
    return temp;
}
```

```
void display(struct Queue *Q)
{
struct Node *temp = Q->front;
while(temp!=NULL)
{
printf("Queue element= %d", temp->data);
temp=temp->next;
}
```